

Lecture 02

AI assistance

Alexis Shakas¹

TA: Ian Wasser²

¹Department of Earth and Planetary Science

²Department of Computer Science

February 24, 2026

Outline

Learning to code

AI assisted coding

Reality check

The big picture

ETH: AI in Teaching and Learning

Where we're at

Lecture	Date	T.H.A.	Room
1: Kickoff	17/2	N	RZ D8
2: AI assistance and tools	24/2	N	RZ D8
3: Project-based teamwork	3/3	N	RZ D8
4: Intro to Git	10/3	N	RZ D8
5: Git continued	17/3	N	TBA
6: Git advanced	24/3	Y	RZ D8
7: AI-assisted Python	31/3	Y	TBA
8: The 3 R's in Python	14/4	Y	TBA
9: Python packaging ecosystem	21/4	N	RZ D8
10: OOP	28/4	Y	RZ D8
11: Testing and debugging	5/5	Y	TBA
12: Joker lecture	12/5	N	RZ D8
13: Project distribution	19/5	N	RZ D8
14: Project marketplace	26/5	N	RZ D8

T.H.A: Take home assignment

TBA: Lecture room to be announced

Today's learning objectives

In today's lecture, we'll focus on the following:

1. Tackle the transition in coding (and why it matters)
2. Explore your (numerous) options for AI assistance
3. Learn practical steps to *not be* a vibe coder
4. Discuss ETH guidelines for AI use

Some reminders

Here are some reminders before we start:

- ▶ Please unenroll by the end of the week if you will not follow the class. Teams will be assigned next week.

Some reminders

Here are some reminders before we start:

- ▶ Please unenroll by the end of the week if you will not follow the class. Teams will be assigned next week.
- ▶ Please briefly outline your project idea by next week (if you have it). Template is on Moodle.

Some reminders

Here are some reminders before we start:

- ▶ Please unenroll by the end of the week if you will not follow the class. Teams will be assigned next week.
- ▶ Please briefly outline your project idea by next week (if you have it). Template is on Moodle.
- ▶ The lecture will be recorded (but still, human presence is expected)

Your preferred learning style

The poll from last week..

- ▶ "Classical" presentation where lecturer covers material and occasionally asks questions - 50%
- ▶ Trying out things regularly during the lecture - 59%
- ▶ Discussing with my peers and having a conversation-like lecture - 59%
- ▶ One "long" presentation session followed by one "long" exercise session - 13%

Outline

Learning to code

AI assisted coding

Reality check

The big picture

ETH: AI in Teaching and Learning

Interacting with machines

- ▶ A **computer program** is an organized list of instructions that make a computer behave in a pre-designed manner.

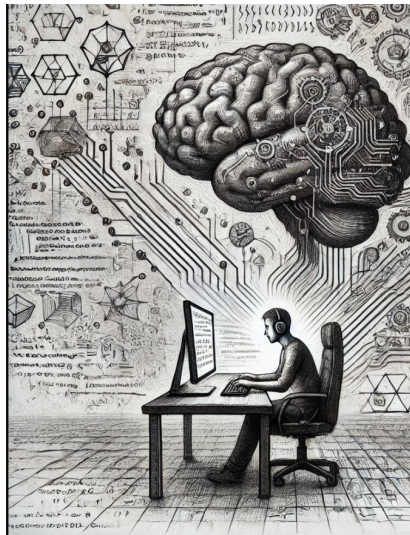


Figure made with AI.

Interacting with machines

- ▶ A **computer program** is an organized list of instructions that make a computer behave in a pre-designed manner.
- ▶ **Computer programming** is the process of designing, writing, testing, debugging, and maintaining computer programs.

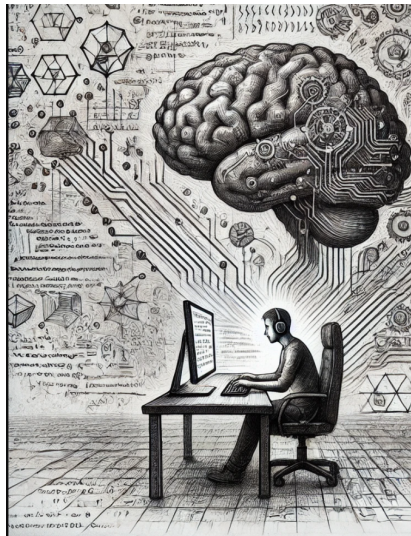


Figure made with AI.

Interacting with machines

- ▶ A **computer program** is an organized list of instructions that make a computer behave in a pre-designed manner.
- ▶ **Computer programming** is the process of designing, writing, testing, debugging, and maintaining computer programs.
- ▶ A **programming language** is a formal set of rules and syntax used to write computer programs.

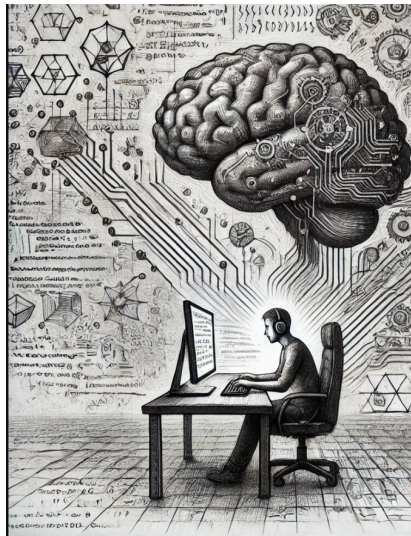
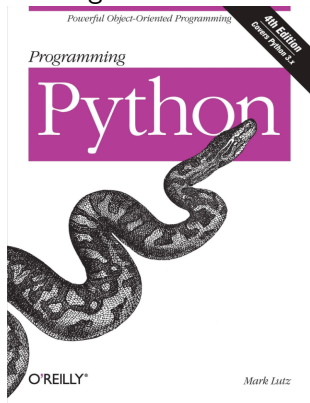


Figure made with AI.

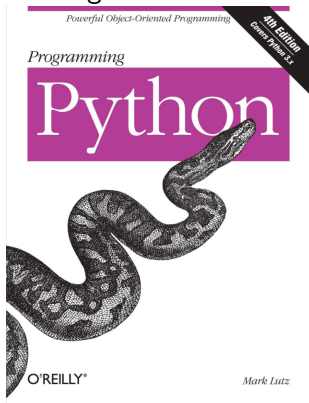
Old-school learning

Coding before 2008



Old-school learning

Coding before 2008



Coding before 2022



Synonyms

`py` `pythonic` `python-shell` `python-interpreter`

Stats

created 16 years, 7 months ago

viewed 141236 times

active 1 month ago

editors 216

A new era of learning to code?



A new era of learning to code?



What is Vibe Coding?

"Vibe coding" is a playful approach to development that relies on high-level prompting. You give broad instructions, accept AI suggestions, and focus on the overall "vibe" of the project rather than implementation details.

What is AI-Assisted Engineering?

This goes beyond vibe coding. AI-assisted engineering is a more structured approach that combines the creativity of vibe coding with the rigor of traditional engineering practices. It involves specs, rigor and emphasizes collaboration between human developers and AI tools, ensuring that the final product is not only functional but also maintainable and secure.

A new era of learning to code?

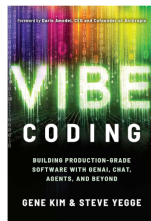


What is Vibe Coding?

"Vibe coding" is a playful approach to development that relies on high-level prompting. You give broad instructions, accept AI suggestions, and focus on the overall "vibe" of the project rather than implementation details.

What is AI-Assisted Engineering?

This goes beyond vibe coding. AI-assisted engineering is a more structured approach that combines the creativity of vibe coding with the rigor of traditional engineering practices. It involves specs, rigor and emphasizes collaboration between human developers and AI tools, ensuring that the final product is not only functional but also maintainable and secure.



MiniTask

Let's (mini) vibe-code! Your task: create a web-based word-frequency analyzer!

1. Go to your favorite LLM (e.g., log in to [Microsoft Copilot](#) with your ETH account)
2. Ask for something along these lines: "create HTML code that I can paste into a file called analyzer.html and that allows me to drop in a text file, it counts and plots the frequency of words in that file"
3. Save the file on your computer as analyser.html
4. If you don't have it from last time, download "Alice in Wonderland" ([available here](#))
5. Drop in the file and look at the plot

Word Frequency Plotter

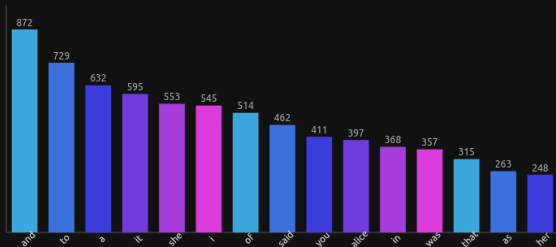
Drop .txt files here and see which words dominate.

Drop text files here
or click to choose files

Top words (excluding common stopwords)

Showing top 15 words

Total words (excluding stopwords): 25693



Done. Adjust "Top N" to change the chart.

Outline

Learning to code

AI assisted coding

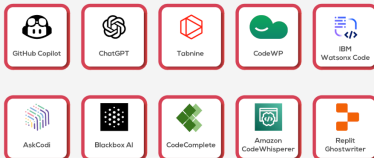
Reality check

The big picture

ETH: AI in Teaching and Learning

Which tool?

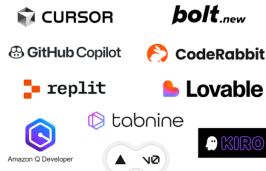
Top Ten AI Tools for Code Generation



5 Best AI tools for Software Developers



AI Tools Every Development Team Should Know



TOP 10 AI TOOLS FOR SOFTWARE DEVELOPMENT



There is no best pick here, and as we said it's more about the "how" than the "what"!

Browser-based LLM coding workflows

Access an LLM remotely on a server..

Typical workflow

1. Open the web app (browser)
2. Write a prompt (and optionally code/files)
3. Iterate on the response (ask for fixes/tests/refactors)
4. Copy code back into your project (or export/share)

Browser-based LLM coding workflows

Access an LLM remotely on a server..

Typical workflow

1. Open the web app (browser)
2. Write a prompt (and optionally code/files)
3. Iterate on the response (ask for fixes/tests/refactors)
4. Copy code back into your project (or export/share)

Strengths / trade-offs

- ▶ + Fastest way to start, no local setup
- ▶ + Easy for learning, brainstorming, and code explanation
- ▶ - No privacy / information accessed from services
- ▶ - Context may be limited unless you attach files/projects
- ▶ - Weaker direct integration with your local repo

Browser-based LLM coding workflows

Some options are..

- ▶ [Apertus](#) for a (purely Swiss) multilingual LLM
- ▶ [DeepSeek](#) for an open-source model
- ▶ [ChatGPT](#) for the most popular LLM
- ▶ [Claude](#) / Claude Code web experiences
- ▶ [Gemini](#) (including Gemini Canvas for code/prototypes)
- ▶ [Microsoft copilot](#) seems to be the default option for ETH
- ▶ Meta AI, perplexity, Grok ...

Browser-based LLM coding workflows

Some options are..

- ▶ [Apertus](#) for a (purely Swiss) multilingual LLM
- ▶ [DeepSeek](#) for an open-source model
- ▶ [ChatGPT](#) for the most popular LLM
- ▶ [Claude](#) / Claude Code web experiences
- ▶ [Gemini](#) (including Gemini Canvas for code/prototypes)
- ▶ [Microsoft copilot](#) seems to be the default option for ETH
- ▶ Meta AI, perplexity, Grok ...

More alternative options

- ▶ [LeChat](#) leading European LLM from Mistral
- ▶ [Lumo](#) for a Swiss-based LLM from ProtonMail
- ▶ [Ecosia](#) for a more ecological alternative
- ▶ [Viro](#) for powering renewable energies
- ▶ [Thaura](#) for an ethical alternative

Any other suggestions? Please share then on Moodle!

Local LLM coding workflows

Install software on your machine, download models and interact locally. Can use command line, apps or web interfaces.

1. Install a local runner / app

- ▶ CLI/runtime: **Ollama**
- ▶ Desktop apps: **Jan**, **LM Studio**
- ▶ Self-hosted web UI: **Open WebUI** (often paired with Ollama)

2. Download / pull a model

- ▶ From the app directly
- ▶ From a website like [Hugging Face](#)

3. Run the model locally

- ▶ Chat in terminal, GUI or your browser

Note that this option relies on your computer's performance!

Strengths / trade-offs and options

Local deployment of LLMs allows:

- ▶ + Max privacy, local file integration
- ▶ + No internet connection needed
- ▶ + Less energy required
- ▶ - Slower than web-based
- ▶ - Usually not the latest models / mostly open models

Strengths / trade-offs and options

Local deployment of LLMs allows:

- ▶ + Max privacy, local file integration
- ▶ + No internet connection needed
- ▶ + Less energy required
- ▶ - Slower than web-based
- ▶ - Usually not the latest models / mostly open models

Some of the most popular options are:

- ▶ [Jan](#) for a user-friendly experience
- ▶ [Kobold.cpp](#) for customization
- ▶ [Ollama](#) for a powerful CLI experience
- ▶ [LM Studio](#) for a server experience
- ▶ [Open WebUI](#) for a browser, web hosted experience

Consider the Swiss-based [apertus](#) LLM!

IDE-based LLM coding workflows

Use AI directly inside your code editor/IDE to get suggestions, chat with your project, and apply edits.

1. **Install your favorite IDE**
2. **Install an IDE extension** if your IDE does not have native AI support
3. **Connect/choose a model**
 - ▶ Sign in to a hosted model (e.g., Github coPilot)
 - ▶ Or connect a local model (beta..)
4. **Interact while coding**
 - ▶ Inline completions as you type
 - ▶ Chat with file/project context
 - ▶ Ask for explanations, refactors, tests, debugging, or multi-file edits

This option is the most convenient for day-to-day coding!

IDE AI assisted workflow

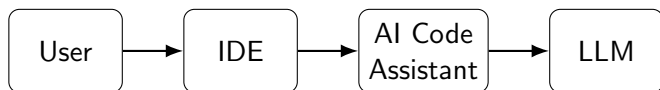


Figure: Linear workflow: the user interacts with the Integrated Development Environment (IDE or editor); the AI coding assistant consults a Large Language Model (LLM) and returns suggestions back through the IDE.

IDE-integrated workflows

Strengths and trade-offs:

- ▶ + Fastest day-to-day workflow (AI where you already code)
- ▶ + Can often connect to both cloud and local models
- ▶ - Usually requires an account/subscription for best features
- ▶ - Project-context features may send code to a remote service (privacy consideration)

IDE-integrated workflows

Strengths and trade-offs:

- ▶ + Fastest day-to-day workflow (AI where you already code)
- ▶ + Can often connect to both cloud and local models
- ▶ - Usually requires an account/subscription for best features
- ▶ - Project-context features may send code to a remote service (privacy consideration)

Some of the most powerful examples are

- ▶ [GitHub Copilot](#) or [Codex](#) extensions for VS Code
- ▶ [JetBrains AI Assistant](#) in PyCharm for a stable choice
- ▶ [Cursor](#) for an AI-native experience
- ▶ [Google Antigravity](#) for agentic development
- ▶ [Kiro](#) for Spec-driven development

MiniTask

Let's move into Python code to make a game you (probably) all know!

1. As before, go to your favorite LLM
2. Ask for something along these lines: "Create a word-guessing (hangman) game in Python that I can paste into a web-based IDE to directly play. Use words from the book 'Alice in Wonderland'" (upload text to LLM).
3. Paste the code in a web-based Python IDE like [Web Tiger Python](#)
4. Can you play the game?

How much of the code can you explain/understand?

Now have a look at the generated code for a few minutes and respond honestly..



Outline

Learning to code

AI assisted coding

Reality check

The big picture

ETH: AI in Teaching and Learning

How AI Impacts Skill Formation

Judy Hanwen Shen, Alex Tamkin



February 3, 2026

Study design

- ▶ **Question:** Does AI coding help developers learn a *new* coding skill?

Study design

- ▶ **Question:** Does AI coding help developers learn a *new* coding skill?
- ▶ **Participants:** 52 software engineers (mostly junior), Python users, unfamiliar with a certain library called **Trio**.

Study design

- ▶ **Question:** Does AI coding help developers learn a *new* coding skill?
- ▶ **Participants:** 52 software engineers (mostly junior), Python users, unfamiliar with a certain library called **Trio**.
- ▶ **Method:** Randomized trial
 - ▶ Control: no AI assistant
 - ▶ Treatment: AI assistant available during coding

Study design

- ▶ **Question:** Does AI coding help developers learn a *new* coding skill?
- ▶ **Participants:** 52 software engineers (mostly junior), Python users, unfamiliar with a certain library called **Trio**.
- ▶ **Method:** Randomized trial
 - ▶ Control: no AI assistant
 - ▶ Treatment: AI assistant available during coding
- ▶ **Flow:** Two Trio coding tasks → quiz on debugging, code reading, writing, and concepts

Study design

- ▶ **Question:** Does AI coding help developers learn a *new* coding skill?
- ▶ **Participants:** 52 software engineers (mostly junior), Python users, unfamiliar with a certain library called **Trio**.
- ▶ **Method:** Randomized trial
 - ▶ Control: no AI assistant
 - ▶ Treatment: AI assistant available during coding
- ▶ **Flow:** Two Trio coding tasks → quiz on debugging, code reading, writing, and concepts
- ▶ **Goal:** Measure both task completion and actual learning/mastery

Main outcomes

Learning outcome

AI assistance reduced mastery on the post-task quiz.

- ▶ **AI group:** $\approx 50\%$ average quiz score
- ▶ **No-AI group:** $\approx 67\%$ average quiz score
- ▶ **Difference: 17 percentage points lower** with AI (statistically significant)

Productivity outcome

AI users finished slightly faster on average, but the speedup was **not statistically significant**.

- ▶ Time savings were partly offset by time spent querying the assistant

Why this happened (interaction patterns mattered)

- ▶ Anthropic identified **6 AI usage patterns**
- ▶ Biggest split was **delegation vs cognitive engagement**

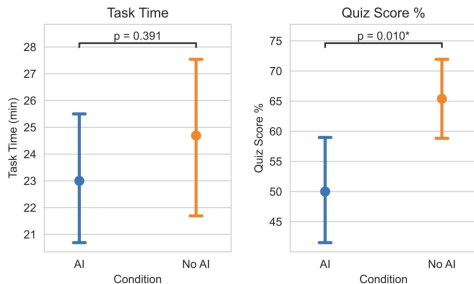
Worse learning	Better learning
Delegating code generation to AI	Asking for explanations / concepts
AI-driven debugging with little self-reasoning	Generate then probe understanding
Progressive reliance on AI	Hybrid code + explanation requests

Takeaway: **Using AI is not the key variable** — **how** developers use it is.

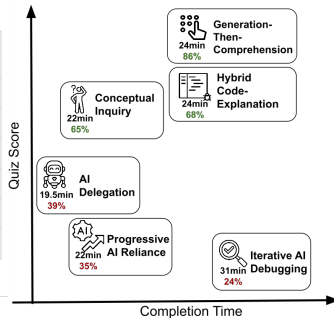
How you use AI matters

The 6 usage patterns identified in the study:

The Impact of AI Assistance on Coding Speed and Knowledge Quiz



AI Usage Patterns



Implications and takeaway

- ▶ AI can help complete unfamiliar coding tasks, but may **weaken skill formation**

Implications and takeaway

- ▶ AI can help complete unfamiliar coding tasks, but may **weaken skill formation**
- ▶ The most affected skills were those needed to **supervise AI safely**:
 - ▶ conceptual understanding
 - ▶ code reading
 - ▶ debugging

Implications and takeaway

- ▶ AI can help complete unfamiliar coding tasks, but may **weaken skill formation**
- ▶ The most affected skills were those needed to **supervise AI safely**:
 - ▶ conceptual understanding
 - ▶ code reading
 - ▶ debugging
- ▶ Practical implication for teams:
 - ▶ Encourage **explanation-seeking** and **conceptual questions**
 - ▶ Avoid full delegation when learning new tools/libraries
 - ▶ **Don't** share code you don't understand; Enforce peer review within the team!

Implications and takeaway

- ▶ AI can help complete unfamiliar coding tasks, but may **weaken skill formation**
- ▶ The most affected skills were those needed to **supervise AI safely**:
 - ▶ conceptual understanding
 - ▶ code reading
 - ▶ debugging
- ▶ Practical implication for teams:
 - ▶ Encourage **explanation-seeking** and **conceptual questions**
 - ▶ Avoid full delegation when learning new tools/libraries
 - ▶ **Don't** share code you don't understand; Enforce peer review within the team!
- ▶ **Bottom line:** AI-enhanced productivity is *not* a shortcut to competence.

MiniTask

Take a few minutes to reflect on your use of AI. Team up with one or more of your classmates, ideally also people you don't know. Discuss the following:

1. How do you currently use AI assistance?
2. How do you check the outcome generated by genAI, if any?
3. What worries do you have about the use of genAI, if any?
4. What are ways in which genAI has helped you in coding?

Outline

Learning to code

AI assisted coding

Reality check

The big picture

ETH: AI in Teaching and Learning

Software (project) design

Since code is easy to produce, your role as a scientific software developer becomes crucial. It is important to structure the project firmly and with a deep understanding before going forward.

Many approaches exist and we'll discuss a few in this class:

- ▶ **Spec-Driven Development (SDD)**

Write a structured *spec* before coding and use that spec as the shared source of truth for both humans and the model.

- ▶ **Domain-Driven Design (DDD)**

Design software around the *domain* (real-world concepts, rules, and language), not just technical layers.

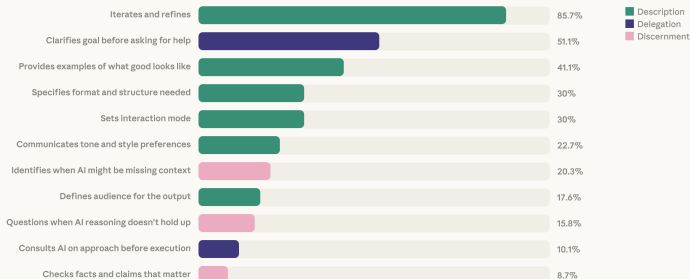
- ▶ **Test-Driven Development (TDD)**

Write tests *before* implementation code, then develop the project in short cycles.

Strive to develop AI fluency

A very recent survey from [Anthropic](#) uses 24 metrics to define AI fluency, and examines these in 9,830 interactions of users with AI code assistants. 11 of these are quantifiable and are shown below.

Behavioral indicator prevalence



Top-down problem solving

In research, we are often faced with open-ended questions. A structured approach to problem solving will not only allow us to better dissect the research hypotheses, but to also create functional software.

The **top-down** problem solving approach consists of:

- ▶ Clearly stating a research questions/goals/objectives.

Top-down problem solving

In research, we are often faced with open-ended questions. A structured approach to problem solving will not only allow us to better dissect the research hypotheses, but to also create functional software.

The **top-down** problem solving approach consists of:

- ▶ Clearly stating a research questions/goals/objectives.
- ▶ Break it into logical components and tasks.

Top-down problem solving

In research, we are often faced with open-ended questions. A structured approach to problem solving will not only allow us to better dissect the research hypotheses, but to also create functional software.

The **top-down** problem solving approach consists of:

- ▶ Clearly stating a research questions/goals/objectives.
- ▶ Break it into logical components and tasks.
- ▶ Revise the logic and further dissect the problem.

Top-down problem solving

In research, we are often faced with open-ended questions. A structured approach to problem solving will not only allow us to better dissect the research hypotheses, but to also create functional software.

The **top-down** problem solving approach consists of:

- ▶ Clearly stating a research questions/goals/objectives.
- ▶ Break it into logical components and tasks.
- ▶ Revise the logic and further dissect the problem.
- ▶ Attempt to solve the problem by solving the individual tasks.

This is parallel to a concept from software engineering called *stepwise refinement*.

Stepwise refinement

Stepwise refinement, also known as incremental development or top-down design, is a programming methodology where a complex problem is broken down into smaller, more manageable pieces (Niklaus Wirth, 1971).

1. Define the main task or goal at a high level without worrying about implementation details.

Stepwise refinement

Stepwise refinement, also known as incremental development or top-down design, is a programming methodology where a complex problem is broken down into smaller, more manageable pieces (Niklaus Wirth, 1971).

1. Define the main task or goal at a high level without worrying about implementation details.
2. Decompose the task into smaller, (logical) simpler subtasks.

Stepwise refinement

Stepwise refinement, also known as incremental development or top-down design, is a programming methodology where a complex problem is broken down into smaller, more manageable pieces (Niklaus Wirth, 1971).

1. Define the main task or goal at a high level without worrying about implementation details.
2. Decompose the task into smaller, (logical) simpler subtasks.
3. Further break down the subtasks into smaller components until each can be implemented directly.

Stepwise refinement

Stepwise refinement, also known as incremental development or top-down design, is a programming methodology where a complex problem is broken down into smaller, more manageable pieces (Niklaus Wirth, 1971).

1. Define the main task or goal at a high level without worrying about implementation details.
2. Decompose the task into smaller, (logical) simpler subtasks.
3. Further break down the subtasks into smaller components until each can be implemented directly.
4. Start by implementing the simplest level and test each part as it's developed. Gradually build up to the complete solution.

Stepwise refinement example: Hangman

1. Define the main goal (high level)

Create a game where the player guesses a hidden word letter by letter, with a limited number of wrong guesses.

Stepwise refinement example: Hangman

1. Define the main goal (high level)

Create a game where the player guesses a hidden word letter by letter, with a limited number of wrong guesses.

2. Decompose into main subtasks

- ▶ Choose a secret word
- ▶ Show game state (blanks, guessed letters, remaining lives)
- ▶ Get and validate user input
- ▶ Update the game state after each guess
- ▶ Check win/lose condition

Stepwise refinement example: Hangman

1. Define the main goal (high level)

Create a game where the player guesses a hidden word letter by letter, with a limited number of wrong guesses.

2. Decompose into main subtasks

- ▶ Choose a secret word
- ▶ Show game state (blanks, guessed letters, remaining lives)
- ▶ Get and validate user input
- ▶ Update the game state after each guess
- ▶ Check win/lose condition

3. Refine each subtask into implementable steps

- ▶ *Choose word*: load a word list → select one word randomly
- ▶ *Display state*: reveal guessed letters, hide others as “_”
- ▶ *Validate input*: one letter, alphabetic, not already guessed
- ▶ *Update state*: add guess to set, reduce lives if incorrect
- ▶ *Check end*: all letters guessed (win) or lives = 0 (lose)

Stepwise refinement example: Hangman

1. Define the main goal (high level)

Create a game where the player guesses a hidden word letter by letter, with a limited number of wrong guesses.

2. Decompose into main subtasks

- ▶ Choose a secret word
- ▶ Show game state (blanks, guessed letters, remaining lives)
- ▶ Get and validate user input
- ▶ Update the game state after each guess
- ▶ Check win/lose condition

3. Refine each subtask into implementable steps

- ▶ *Choose word*: load a word list → select one word randomly
- ▶ *Display state*: reveal guessed letters, hide others as “_”
- ▶ *Validate input*: one letter, alphabetic, not already guessed
- ▶ *Update state*: add guess to set, reduce lives if incorrect
- ▶ *Check end*: all letters guessed (win) or lives = 0 (lose)

4. Implement and test incrementally

- ▶ First: word selection + simple display
- ▶ Then: input handling and validation, guess updates and lives
- ▶ Finally: game loop, win/lose messages, replay option

MiniTask

Apply stepwise refinement alone or in a group to create the game "Anagrams": where a word is scrambled and the user has to find the right sequence. For example, "NSHWIACD" is an anagram for "SANDWICH". You can use "Alice in Wonderland" to extract word choices.

1. How well can you define the goal of the game?
2. How many subtasks can you think of?
3. Can you think of tests for the software?

Hint: If you write your refinement skeleton in [Markdown](#) makes it very easy for LLMs to read and follow. If you don't know what Markdown is, don't worry; plain text also works.

MiniTask

Supply your stepwise refinement text to your favorite LLM and instruct it to create a Python version of the game.

1. Ask for well documented code
2. As for only standard Python libraries
3. Run the code in (e.g.) Web Tiger Python to check if it works.

Finally, read through the code. How much of it do you understand now compared to the previous example?

Outline

Learning to code

AI assisted coding

Reality check

The big picture

ETH: AI in Teaching and Learning

ETH's stance on genAI use

ETH promotes responsible usage of generative AI in education, encouraging ingenuity and innovative approaches while prioritising responsibility, transparency, and fairness by adhering to privacy and copyright regulations.

- ▶ support lecturers and students in acquiring advanced skills for working with generative AI by offering targeted courses.

ETH's stance on genAI use

ETH promotes responsible usage of generative AI in education, encouraging ingenuity and innovative approaches while prioritising responsibility, transparency, and fairness by adhering to privacy and copyright regulations.

- ▶ support lecturers and students in acquiring advanced skills for working with generative AI by offering targeted courses.
- ▶ promote the development and embedding of AI technologies in teaching and learning as well as for academic and administrative work.

ETH's stance on genAI use

ETH promotes responsible usage of generative AI in education, encouraging ingenuity and innovative approaches while prioritising responsibility, transparency, and fairness by adhering to privacy and copyright regulations.

- ▶ support lecturers and students in acquiring advanced skills for working with generative AI by offering targeted courses.
- ▶ promote the development and embedding of AI technologies in teaching and learning as well as for academic and administrative work.
- ▶ equipping students and staff for the AI-supported labour market and societies of tomorrow.

Recommended Generative AI Tools

ETH recommends tools accessible via **ETH Cloud Subscriptions**, ensuring data-protected access where personal data is not used for training.

- ▶ **Microsoft Copilot:** Free text/image generation via M365 subscription.
- ▶ **Google Gemini:** Based on Google LLMs; supports text, image, and voice input.
- ▶ **Google NotebookLM:** Designed for interacting with personal resources and generating podcasts.

Visit [ETH AI Tools Page](#) for more information.

Practical tips from ETH

The **Do's**:

- ▶ **Prompting:** Be clear, provide context, and define the AI's role.
- ▶ **Data Minimization:** Only provide necessary information.
- ▶ **Disable History:** Use settings to prevent data from being reused for model training.
- ▶ **Critical Review:** Always check for *hallucinations* and verify sources.

Practical tips from ETH

The **Do's**:

- ▶ **Prompting:** Be clear, provide context, and define the AI's role.
- ▶ **Data Minimization:** Only provide necessary information.
- ▶ **Disable History:** Use settings to prevent data from being reused for model training.
- ▶ **Critical Review:** Always check for *hallucinations* and verify sources.

The **Dont's**:

- ▶ **No Confidential Info:** Do not share (strictly) confidential or proprietary information.
- ▶ **No Personal Data:** Avoid uploading names, ID numbers, or sensitive health/political data.
- ▶ **Don't Blindly Rely:** Watch for lack of coherence, repetitions, and semantic errors.
- ▶ **No Illegal Acts:** Do not use AI for discriminatory, harassing, or fraudulent activities.

Copyright and Data Security

- ▶ **Copyright:** You take responsibility for generated content and will have to sign “declaration of originality”.
- ▶ **FADP (Swiss Act):** Principles of transparency, data minimization, and right to information.
- ▶ **EU AI Act:** Risk-based approach; relevant if data/outputs involve the EU.

Copyright and Data Security

- ▶ **Copyright:** You take responsibility for generated content and will have to sign “declaration of originality”.
- ▶ **FADP (Swiss Act):** Principles of transparency, data minimization, and right to information.
- ▶ **EU AI Act:** Risk-based approach; relevant if data/outputs involve the EU.

ETH data classification levels:

Public/Internal: General business data.

Confidential: May only be processed in **approved** external cloud services.

Strictly Confidential: **Prohibited** in any external cloud service.

Strictly Prohibited: Sharing personal research data, strategic financial documents, or security-sensitive data with public AI.

Recap and summary

Here are some take-home thoughts:

1. How does LLM use affect your learning experience?
2. How do you plan to "stay in control" of your project?
3. What tools do you plan to use (this may change!) ?

Recap and summary

Here are some take-home thoughts:

1. How does LLM use affect your learning experience?
2. How do you plan to "stay in control" of your project?
3. What tools do you plan to use (this may change!) ?

Finally, if you decide to stay in the class, you have to fill out a short but **mandatory** questionnaire on Moodle at the end of the week, to turn in by Monday morning. This will help me assign you to teams.